

Desenvolvimento de um Assistente de Pesquisa Inteligente para E-Commerce:**Implementação, Resultados e Avaliação Crítica**

Workshop 2 - Relatório

Verónica Vitorino (N.º 57699) e Miguel Grilo (N.º 58387)

Resumo: Este relatório descreve a implementação do Assistente de Pesquisa Inteligente para E-Commerce em Português de Portugal, evoluindo da base conceitual apresentada no Workshop 1 para um sistema em produção assente em .NET 8 e Azure Functions. São apresentadas a arquitetura do sistema, os modelos de IA integrados no pipeline, os resultados com base em métricas operacionais e experimentais, e uma análise crítica das limitações, vieses e oportunidades de melhoria identificadas ao longo do desenvolvimento.

1 - Evolução face ao Workshop 1

O Workshop 1 estabeleceu a fundamentação matemática do sistema. O Workshop 2 reporta a sua concretização num sistema funcional e implantável. A tabela seguinte sintetiza as principais correspondências entre teoria e implementação real.

Workshop 1 (Teoria)	Workshop 2 (Implementação)
BM25, Cosseno, RRF e Transformer como fundamento conceptual	Azure AI Search aplica BM25 (BM25Similarity) e HNSW com métrica cosseno; fusão híbrida delegada ao serviço (equivalente ao RRF)
LLM extrai JSON com filtros OData	Pipeline de 3 camadas: (1) reformulação LLM, (2) pesquisa híbrida + filtros OData, (3) segundo LLM de validação de relevância
Catálogo com CatalogRules	QueryReformulatorPrompt + SearchAttributesIndexFilter com mapeamento de tamanhos, género, preço e cores
Modelo gpt-4.1-nano (conceito)	Modelo gpt-5.4-mini em produção (QueryConstants) — equilíbrio latência/capacidade

A mensagem central desta evolução é a transição de um modelo matemático para um sistema em produção com orquestração, resiliência (retries, round-robin de embeddings) e controlo de qualidade pós-recuperação. A fundamentação matemática mantém-se; a implementação usa os serviços geridos que aplicam esses algoritmos.

2 - Arquitetura do Sistema

O sistema está organizado em três projetos .NET 8: Ecommerce.ChatAssistant.Api (Azure Functions: chat, index-data, process-data), Ecommerce.ChatAssistant.Application (serviços de LLM, embeddings, pesquisa e indexação) e Ecommerce.ChatAssistant.Custom (prompts, mappers, filtros OData, constantes).

2.1 - Pipeline de Ingestão

A ingestão de produtos segue um fluxo assíncrono com as seguintes etapas:

1. JSON de produtos submetido ao endpoint HTTP index-data e guardado num blob json-data-receiver;
2. Timer process-data (1 min) processa o blob invocando ProductSearchDocumentMapper.BuildEmbeddedContent — concatenação de título, descrição, marca, categoria e materiais em minúsculas;
3. Antes da transformação e geração de embeddings, o pipeline executa uma barreira de validação estrutural através do componente ProcessDataHelper. Cada produto é verificado quanto à presença de chaves obrigatórias, integridade de variantes e tipos de dados esperados no schema normalizado. Qualquer desconformidade ou dado inválido interrompe imediatamente o processamento do blob, lançando uma exceção com o registo do SKU e o respetivo motivo. Esta abordagem rigorosa previne a corrupção silenciosa e a indexação parcial de dados defeituosos no índice.
4. Embeddings gerados em lotes de 100 via EmbeddingService (round-robin sobre 5 deployments de text-embedding-3-small, dimensão 1536);
5. Documentos indexados no índice commerce-index-pt-pt do Azure AI Search;
6. Blob movido para completed ou error conforme o resultado.

2.2 - Pipeline de Chat

Otimização de Custos e Intercepção Precoce (Phase 1 / Phase 2): Por razões de eficiência e controlo de custos computacionais (redução de chamadas à API de Embeddings e ao Azure AI Search), o fluxo de inferência adota uma arquitetura em duas fases. A "Phase 1" atua como uma barreira de segurança e intenção: valida a existência de payload e interceta conversas genéricas (saudações) ou fora do domínio de moda (out-of-scope). Apenas as queries estritamente transacionais transitam para a "Phase 2", que despoleta o pipeline completo de NLP e pesquisa híbrida.

O fluxo de inferência compreende cinco etapas sequenciais:

1. **Reformulação** — Reformulação LLM: o QueryReformulatorService devolve um JSON com reformulated_query, search_attributes (género, marca, preço, categoria, cor, tamanho) e flags needs_more_context / facet_lookup.
2. **Embedding** — Embedding da query: se a reformulação for válida, o EmbeddingService vetoriza a query; caso contrário, a pesquisa degrada para modo texto com operadores fuzzy/wildcard Lucene (*~).
3. **Pesquisa híbrida** — Pesquisa híbrida: o HybridSearchService submete ao Azure AI Search texto + vetor. A filtragem OData é orquestrada pelo SearchAttributesIndexFilter, que aplica uma abordagem dual: parâmetros absolutos (como gender, audience, season) utilizam o operador determinístico eq; em contrapartida, atributos suscetíveis a variações lexicais (como brand, color) recorrem à função nativa search.ismatch, que executa um full-text match tolerante a desvios ortográficos. Adicionalmente, o atributo pattern implementa normalização de sinónimos, e as restrições financeiras aplicam uma lógica de intersecção de intervalos (priceMax ge min_price e priceMin le max_price), assegurando que artigos com variantes de diferentes preços não sejam erradamente excluídos.
4. **Score mínimo** — Filtragem por score mínimo: limiar ProductMinScore = 0.008, retendo até 100 candidatos.
5. **Validação** — Validação LLM: o SearchRelevanceValidatorService devolve os IDs relevantes; apresentados no máximo 20 produtos. Se ≥ 20 candidatos forem relevantes, o sistema emite uma pergunta de refinamento.

Decisão Final e Controlo de Broad Search: A lógica do ChatFunction encapsula a decisão final da resposta. Perante pesquisas demasiado amplas que retornam um número excessivo de candidatos, o sistema adota uma resposta híbrida controlada por um teto de visualização (ChatMaxProductsDisplayed = 20). Paralelamente, o SearchRelevanceValidatorService deteta o cenário de pesquisa ampla e utiliza um prompt especializado (RefinementQuestionSystemPrompt) para gerar uma pergunta de refinamento cirúrgica, focada na extração de um único atributo em falta, evitando bloqueios na navegação e contagens incoerentes na interface.

Fluxo de Exploração (Facet Lookup): Como alternativa à pesquisa de produtos, o pipeline suporta consultas exploratórias ("que tamanhos têm para calças?"). Quando o LLM infere esta intenção, extrai o facet_field alvo e ativa a flag facet_lookup. O fluxo desvia-se do motor de pesquisa tradicional e aciona o FacetSearchService, que interroga diretamente o índice do Azure AI Search aplicando os filtros de contexto vigentes. O sistema devolve unicamente os valores de facet disponíveis, proporcionando uma experiência conversacional fluida e orientada à descoberta do inventário.

Resiliência e Memória Multi-turno (Context Fallback): A gestão de estado conversacional evita o reenvio dispendioso de todo o histórico de mensagens ao LLM. Em vez disso, o sistema atualiza um context_summary a cada turno. Para garantir que não há perda silenciosa de intenção em diálogos longos, foi implementada uma cascata de fallback estrita na construção do contexto atual (ContextSummary → Context do Request → ReformulatedQuery anterior → Query original). Mais ainda, este currentContext consolidado é injetado como prefixo no texto submetido ao Azure AI Search, o que força a representação vetorial a carregar a "memória semântica" da conversa, garantindo que o algoritmo HNSW recupere produtos coerentes com o histórico do diálogo.

2.3 - Configuração do Índice

O índice incorpora um perfil de scoring productBoost com pesos de título (20) e descrição (10). A pesquisa vetorial usa HNSW com efSearch=300 e métrica cosseno. O campo embedding (vetorial) é separado do campo searchText (léxico), preservando a independência de ambos os paradigmas estabelecidos no Workshop 1.

3 - Resultados e Métricas

3.1 Caracterização do Catálogo

O catálogo de demonstração contém 2004 produtos de moda em português de Portugal, distribuídos por 11 marcas. A distribuição de categorias é heterogénea: categorias como casual (~500 produtos) dominam em relação a categorias de nicho, como calçado (~174 produtos), o que produz viés de IDF/BM25 — termos frequentes em categorias dominantes obtêm menor peso.

3.2 Análise Qualitativa por Tipo de Consulta

Com base nas interações realizadas durante o desenvolvimento, identificaram-se quatro padrões de comportamento distintos do sistema:

- Para **consultas semânticas** sem sobreposição lexical direta com o catálogo (e.g., "*algo quente e casual para o inverno*"), a pesquisa híbrida demonstrou ser claramente superior à pesquisa exclusivamente textual: o embedding da query reformulada pelo LLM permite recuperar artigos como casacos polares ou fleece que não partilham nenhum termo com a consulta original.
- Para **consultas lexicais** com marca ou referência explícita (e.g., "*GANT polo*"), o BM25 combinado com o scoring profile (peso título = 20) domina o ranking, devolvendo resultados precisos independentemente do caminho vectorial.
- Para **consultas estruturadas** com atributos explícitos (e.g., "*casaco feminino azul até 80€*"), os filtros OData gerados pelo LLM revelaram-se determinantes: sem eles, o índice devolve candidatos fora da faixa de preço ou género, aumentando significativamente o ruído nos resultados.
- Para **consultas de exploração** (e.g., "*que cores há em sapatinhas?*"), o fluxo de facet lookup activa o FacetSearchService em vez do pipeline de pesquisa, devolvendo os valores disponíveis no catálogo sem listar produtos — comportamento correcto e distinto dos restantes cenários.

3.3 Métricas Operacionais Disponíveis

O sistema expõe, em cada resposta, um conjunto de indicadores que permitem monitorizar o comportamento do pipeline sem necessidade de ground truth anotado:

- **@search.score** por produto — permite observar a distribuição de scores e o efeito do scoring profile por tipo de consulta;
- **count** de produtos devolvidos — indicador de recall operacional;
- **UsedHybrid** (interno) — regista se o pedido activou o caminho híbrido (texto + vector) ou apenas o caminho textual com fuzzy/wildcard;
- **Presença de search_attributes** na reformulação — indica a qualidade da extracção estruturada pelo LLM;
- **Taxa de needs_more_context** — proporção de consultas demasiado vagas para pesquisa directa;
- **Taxa de facet_lookup** — proporção de consultas de exploração de catálogo.

3.4 Proposta de Avaliação Quantitativa

A ausência de um conjunto de teste anotado com ground truth constitui a principal limitação da avaliação nesta fase. Para uma avaliação rigorosa, propõe-se o seguinte protocolo: construção de um conjunto de 30 a 50 consultas em português de Portugal, cobrindo os quatro tipos identificados na secção 3.2, com anotação binária (relevante / não relevante) dos produtos devolvidos no top-10. Sobre este conjunto, calcular-se-iam as métricas Precision@5, Precision@10, MRR (*Mean Reciprocal Rank*) e nDCG@10, comparando as configurações híbrida, só texto, com e sem filtros OData, e com e sem validador LLM. Este protocolo está identificado como trabalho futuro prioritário, sendo a coleção Postman, disponível no repositório, o ponto de partida natural para a sua implementação.

4 - Demonstração do Sistema

Esta secção apresenta exemplos concretos de interação com o assistente, ilustrando os diferentes fluxos de execução do pipeline. Os exemplos incluem a consulta do utilizador, a reformulação produzida pelo LLM, os filtros OData gerados e a resposta final do sistema.

4.1 Cenários de Teste

A tabela seguinte descreve os cinco cenários de demonstração cobrindo os principais fluxos do sistema:

Cenário	Query do utilizador	Comportamento esperado do sistema
1 — Semântico	"Quero algo quente e casual para o inverno"	LLM reformula para termos como casaco/polar; embedding gerado; pesquisa híbrida devolve artigos de vestuário exterior relevantes
2 — Lexical/Marca	"GANT polo azul"	BM25 + scoring profile dominam; filtro OData de cor aplicado; validador confirma relevância
3 — Estruturado com preço	"Casaco feminino até 80 euros"	LLM extrai gender=Feminino, max_price=80; filtro OData composto reduz candidatos; resultados ordenados por score
4 — Facet lookup	"Que cores há disponíveis em sapatilhas?"	LLM ativa flag facet_lookup; FacetSearchService devolve lista de valores de cor sem listar produtos
5 — Contexto insuficiente	"Preciso de um presente"	LLM activa needs_more_context; sistema pede clarificação ao utilizador sem executar pesquisa

4.2 Fluxo de Conversa — Consulta Semântica

O seguinte exemplo ilustra o fluxo completo para uma consulta semântica sem sobreposição lexical direta com o catálogo.

Utilizador: "Quero algo quente e casual para o inverno, masculino, até 100 euros"

Reformulação LLM (QueryReformulatorService):

```
{
  "valid": true,
  "reformulated": "quente casual inverno homem",
  "needs_more_context": false,
  "message": "Selecionei para si opções quentes e casuais para o inverno, com perfil masculino e dentro do orçamento até 100 euros. Posso afinar a pesquisa por casacos, camisolas ou sweatshirts — prefere alguma destas peças?",
  "context_summary": "O utilizador procura roupa quente e casual para inverno, masculino, com orçamento até 100 euros.",
  "search_attributes": {
    "gender": ["Homem"],
    "season": ["Outono/Inverno"],
    "max_price": 100,
    "intent": "product_search",
    "facet_field": ""
  }
}
```

Screenshot Postman — Resposta do endpoint /chat:

```
[ RESPOSTA DA API ]
Mensagem do Assistente: "Selecionei para si opções quentes e casuais para o inverno, com perfil masculino e dentro do orçamento até 100 euros. Posso afinar a pesquisa por casacos, camisolas ou sweatshirts — prefere alguma destas peças?"

Total de Resultados: 15
Top 1 Produto: "Casaco Polar em Fleece"
Preço: 75,50 EUR (Respeita o limite de 100€)
Categoria: Casaco
@search.score: 0.0345

[ NOTA DE COMPORTAMENTO SEMÂNTICO ]
O sistema recuperou produtos de inverno (como o material "Polar" e "Fleece") através da proximidade geométrica dos vetores aos conceitos "quente" e "inverno", contornando a ausência destas palavras-chave exatas no título e descrição do produto.
```

4.3 - Fluxo de Conversa - Consulta com Filtros Estruturados

Exemplo de consulta em que o LLM extrai múltiplos atributos estruturados, que são convertidos em um filtro OData composto.

Utilizador: "Casaco feminino azul até 80 euros"

Filtro OData gerado:

```
status eq 'I'
and gender eq 'Feminino'
and price le 80.0
and search.in(color, 'Azul,Navy,Blue', '')
```

Screenshot Postman — Comparação com/sem filtros OData:

```
[ EXECUÇÃO COM FILTROS ODATA ATIVOS ]  
Total de Resultados: 3  
Top 1 Produto: "GANT Casaco Liso"  
Preço: 51,04 EUR (Respeita o orçamento exigido)  
@search.score: 0.0322  
  
[ EXECUÇÃO SEM FILTROS ODATA (Apenas Texto + Vetor) ]  
Total de Resultados: 8  
Top 1 Produto: "GANT Casaco Xadrez"  
Preço: 88,36 EUR (Ignora o limite de 80€ exigido)  
@search.score: 0.0325
```

4.4 Fluxo de Conversa - Facet Lookup

Exemplo do fluxo de exploração de catálogo em que o sistema não devolve produtos, mas sim os valores disponíveis para um atributo.

Utilizador: “Que cores há disponíveis em sapatilhas?”

Screenshot Postman — Resposta de facet lookup:

```
intent eq 'facet_lookup'  
and facet_field eq 'color'  
and category eq 'Sapatilhas'  
facets: ['Amarelo', 'Antracite', 'Azul', 'Azul-marinho', 'Bege', 'Bordeaux', 'Branco', 'Castanho', 'Cinzento', 'Dourado', 'Laranja',  
'Lilás', 'Preto', 'Rosa', 'Verde', 'Verde-azeitona', 'Vermelho']
```

5 - Análise Crítica

5.1 - Limitações

- **Dependência de plataforma:** Dependência de serviços geridos pelo Azure (Search, OpenAI, Storage) — impossibilita reprodução offline e introduz custo por pedido;
- **Latência dupla:** Duas chamadas LLM por pedido de chat aumentam latência e custo; falhas caem em fallback (validador devolve todos os IDs ordenados), preservando disponibilidade, mas sacrificando precisão;
- **Heurísticas não calibradas:** ProductMinScore = 0.008 e limite de 20 produtos são heurísticas empíricas, não otimizadas estatisticamente;
- **Normalização de texto:** embedded_content gerado em minúsculas pode atenuar nuances léxicas na representação vectorial;
- **Escala do catálogo:** Catálogo de demonstração (~2000 itens) limita a generalização à moda PT — marcas desbalanceadas introduzem viés de popularidade no IDF.

5.2 Viés Identificados

- **Viés de popularidade:** Categorias dominantes no índice penalizam termos frequentes via IDF, podendo desfavorecer produtos de nicho;
- **Scoring profile:** O peso elevado do título (20) no scoring profile penaliza produtos com títulos pouco descritivos face a produtos com descrições ricas;
- **Viés do LLM:** O LLM pode inferir incorretamente atributos de género ou faixa etária a partir de linguagem ambígua (e.g., "para o meu filho") sem ativar needs_more_context.

6 - Conclusão

Do ponto de vista da continuidade científica, o sistema constitui uma instância concreta do paradigma de recuperação de informação híbrida aplicado ao e-commerce em língua portuguesa — um domínio sub-representado na literatura de IR, com potencial para contribuição original através de benchmarks públicos e análise de qualidade de embeddings multilíngues em contexto comercial.